

Git Repository Link: <https://github.com/amitchell-thomson/cwm-fdi>

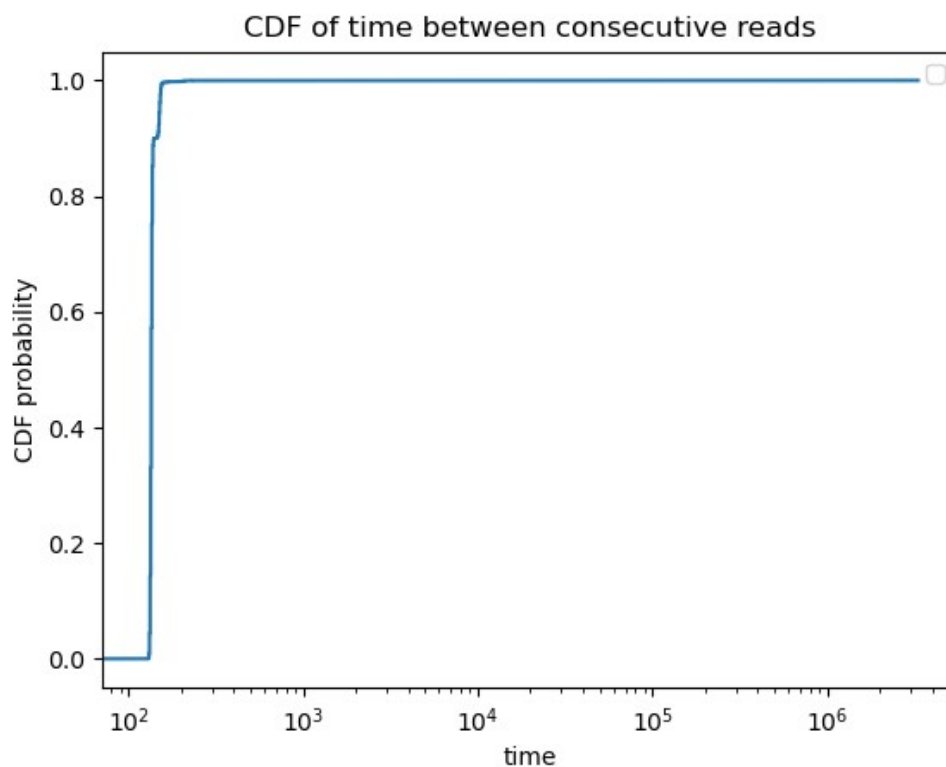
Reading Timestamp Counter

1.

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ for _ in $(seq 1 10); do python3 rdtime.py; done
CPU time counter: 258094371916056 ns
CPU time diff: 16156 ns
CPU time counter: 258094391936360 ns
CPU time diff: 14197 ns
CPU time counter: 258094411085252 ns
CPU time diff: 14830 ns
CPU time counter: 258094430163843 ns
CPU time diff: 14026 ns
CPU time counter: 258094449323719 ns
CPU time diff: 15014 ns
CPU time counter: 258094468432015 ns
CPU time diff: 15485 ns
CPU time counter: 258094487558793 ns
CPU time diff: 15404 ns
CPU time counter: 258094506844919 ns
CPU time diff: 14070 ns
CPU time counter: 258094525980431 ns
CPU time diff: 15277 ns
CPU time counter: 258094545203070 ns
CPU time diff: 14578 ns
```

time gap between consecutive reads sits at around 15000ns (15 microseconds) across a range of around 2 nanoseconds

2.



3.

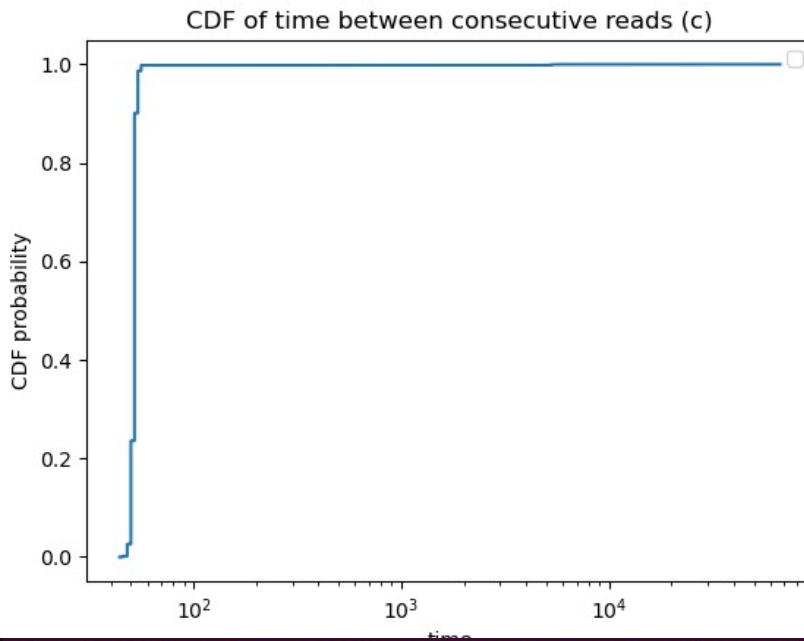
```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ for i in $(seq 1 10); do ./rdtime; done
CPU frequency: 0.900 GHz
Minimum consecutive TSC diff: 190 cycles
Approximate time: 211.04 ns
CPU frequency: 0.900 GHz
Minimum consecutive TSC diff: 188 cycles
Approximate time: 208.86 ns
CPU frequency: 0.900 GHz
Minimum consecutive TSC diff: 248 cycles
Approximate time: 275.58 ns
CPU frequency: 0.900 GHz
Minimum consecutive TSC diff: 248 cycles
Approximate time: 275.51 ns
CPU frequency: 0.900 GHz
Minimum consecutive TSC diff: 252 cycles
Approximate time: 280.00 ns
CPU frequency: 0.900 GHz
Minimum consecutive TSC diff: 182 cycles
Approximate time: 202.22 ns
CPU frequency: 0.900 GHz
Minimum consecutive TSC diff: 260 cycles
Approximate time: 288.89 ns
CPU frequency: 0.900 GHz
Minimum consecutive TSC diff: 276 cycles
Approximate time: 306.67 ns
CPU frequency: 0.900 GHz
Minimum consecutive TSC diff: 266 cycles
Approximate time: 295.54 ns
CPU frequency: 0.900 GHz
Minimum consecutive TSC diff: 246 cycles
Approximate time: 273.33 ns
```

4.

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ for i in $(seq 1 10); do ./rdtime; done
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 68 cycles
Approximate time: 18.89 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 64 cycles
Approximate time: 17.78 ns
CPU frequency: 3.600 GHz
Minimum consecutive TSC diff: 62 cycles
Approximate time: 17.22 ns
```

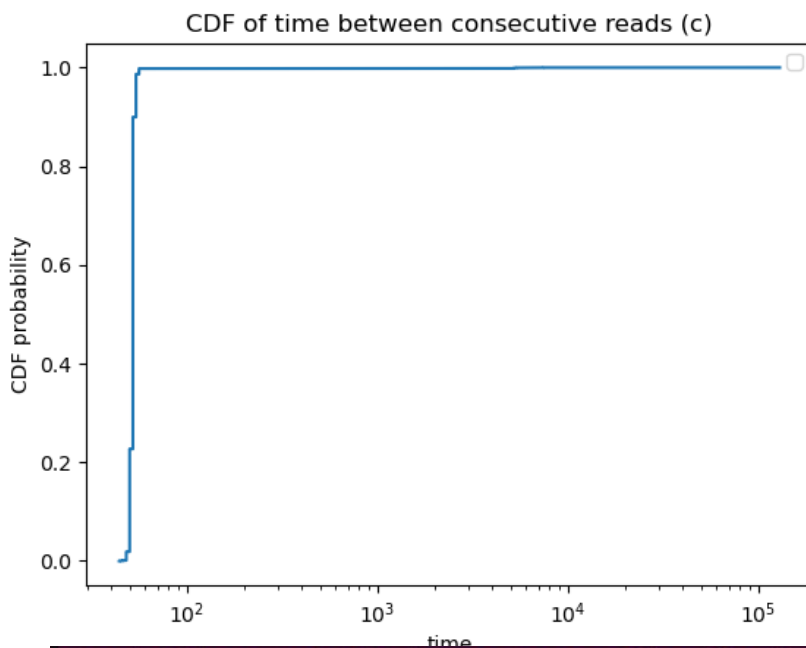
performance mode massively increased clock speed, and drastically reduced timed difference between reads.

5.



```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 ../scripts/plot.py
min/max: 44.0/86742.0
mean: 63.19122519122519
median: 52.0
90th percentile: 54.0
99th percentile: 56.0
```

6. Now pinned to a single core



```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ python3 ../scripts/plot.py
min/max: 44.0/129278.0
mean: 63.15899115899116
median: 52.0
90th percentile: 54.0
99th percentile: 56.0
```

7. Looking at the two cases, pinning the program to one core vs not, we see the only real major difference is the max time taken, which is much larger (we also see a very slight mean time reduction). This could be because one core is subject to higher variance as other processes could get in the way, whereas on multiple cores the blockers are spread out so time difference is less affected by small local changes.

8. The mean times for the c code are much faster than the python code, but their min times are within the same order of magnitude – even close. This is an interesting result shows the way in which python has systematic execution overhead that gives similar best case scenarios to C, but much worse general execution times. Some reasons for this in python are garbage pauses, global interpreter lock contention, interpreter overhead and less efficient machine code than C.

9. The huge difference between mean and maximum measured times come from the rare tail events that occasionally occur as the code runs. These would be things like hardware interrupts and garbage collection (mentioned above), that essentially pause the running program and cause a spike in execution time for that iteration.

Ping Measurement

10.

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ping 8.8.8.8 -c 100 -i 0.2
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=113 time=3.31 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=113 time=3.43 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=113 time=3.36 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=113 time=3.28 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=113 time=4.37 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=113 time=3.22 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=113 time=3.34 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=113 time=3.29 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=113 time=3.27 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=113 time=3.28 ms
^C
--- 8.8.8.8 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1806ms
rtt min/avg/max/mdev = 3.222/3.414/4.367/0.322 ms
```

11.

Oxford

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ping www.ox.ac.uk -c 100 -i 0.2
PING www.ox.ac.uk.cdn.cloudflare.net (104.20.34.13) 56(84) bytes of data.
64 bytes from 104.20.34.13: icmp_seq=1 ttl=52 time=3.83 ms
64 bytes from 104.20.34.13: icmp_seq=2 ttl=52 time=3.96 ms
64 bytes from 104.20.34.13: icmp_seq=3 ttl=52 time=3.82 ms
64 bytes from 104.20.34.13: icmp_seq=4 ttl=52 time=3.71 ms
64 bytes from 104.20.34.13: icmp_seq=5 ttl=52 time=4.03 ms
64 bytes from 104.20.34.13: icmp_seq=6 ttl=52 time=3.86 ms
64 bytes from 104.20.34.13: icmp_seq=7 ttl=52 time=3.98 ms
64 bytes from 104.20.34.13: icmp_seq=8 ttl=52 time=3.97 ms
64 bytes from 104.20.34.13: icmp_seq=9 ttl=52 time=3.94 ms
64 bytes from 104.20.34.13: icmp_seq=10 ttl=52 time=3.98 ms
^C
--- www.ox.ac.uk.cdn.cloudflare.net ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1807ms
rtt min/avg/max/mdev = 3.705/3.908/4.032/0.094 ms
```

Stanford

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ping www.stanford.edu -c 100 -i 0.2
PING pantheon-systems.map.fastly.net (151.101.66.133) 56(84) bytes of data
64 bytes from 151.101.66.133: icmp_seq=1 ttl=54 time=3.32 ms
64 bytes from 151.101.66.133: icmp_seq=2 ttl=54 time=3.43 ms
64 bytes from 151.101.66.133: icmp_seq=3 ttl=54 time=3.32 ms
64 bytes from 151.101.66.133: icmp_seq=4 ttl=54 time=3.49 ms
64 bytes from 151.101.66.133: icmp_seq=5 ttl=54 time=3.49 ms
64 bytes from 151.101.66.133: icmp_seq=6 ttl=54 time=3.45 ms
64 bytes from 151.101.66.133: icmp_seq=7 ttl=54 time=3.48 ms
64 bytes from 151.101.66.133: icmp_seq=8 ttl=54 time=3.40 ms
64 bytes from 151.101.66.133: icmp_seq=9 ttl=54 time=3.38 ms
64 bytes from 151.101.66.133: icmp_seq=10 ttl=54 time=3.35 ms
^C
--- pantheon-systems.map.fastly.net ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1808ms
rtt min/avg/max/mdev = 3.318/3.409/3.489/0.064 ms
```



```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ping www.nus.edu.sg -c 100 -i 0.2
PING mgnzsqc.x.incapdns.net (45.60.33.225) 56(84) bytes of data.
64 bytes from 45.60.33.225: icmp_seq=1 ttl=50 time=3.31 ms
64 bytes from 45.60.33.225: icmp_seq=2 ttl=50 time=3.42 ms
64 bytes from 45.60.33.225: icmp_seq=3 ttl=50 time=3.35 ms
64 bytes from 45.60.33.225: icmp_seq=4 ttl=50 time=3.35 ms
64 bytes from 45.60.33.225: icmp_seq=5 ttl=50 time=4.58 ms
64 bytes from 45.60.33.225: icmp_seq=6 ttl=50 time=3.28 ms
64 bytes from 45.60.33.225: icmp_seq=7 ttl=50 time=3.42 ms
64 bytes from 45.60.33.225: icmp_seq=8 ttl=50 time=3.42 ms
64 bytes from 45.60.33.225: icmp_seq=9 ttl=50 time=3.33 ms
64 bytes from 45.60.33.225: icmp_seq=10 ttl=50 time=3.43 ms
^C
--- mgnzsqc.x.incapdns.net ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1808ms
rtt min/avg/max/mdev = 3.283/3.490/4.581/0.367 ms
```

12. Lab machine to itself 10 times

```
ubuntu@ubuntu:~/CWM-FDI/assignment2$ ping 127.0.0.1 -c 100 -i 0.2
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.020 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.016 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.019 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.019 ms
64 bytes from 127.0.0.1: icmp_seq=5 ttl=64 time=0.018 ms
64 bytes from 127.0.0.1: icmp_seq=6 ttl=64 time=0.019 ms
64 bytes from 127.0.0.1: icmp_seq=7 ttl=64 time=0.020 ms
64 bytes from 127.0.0.1: icmp_seq=8 ttl=64 time=0.019 ms
64 bytes from 127.0.0.1: icmp_seq=9 ttl=64 time=0.018 ms
64 bytes from 127.0.0.1: icmp_seq=10 ttl=64 time=0.016 ms
^C
--- 127.0.0.1 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 1869ms
rtt min/avg/max/mdev = 0.016/0.018/0.020/0.001 ms
```

13. Lab machine to itself 100 times at 0.01s interval

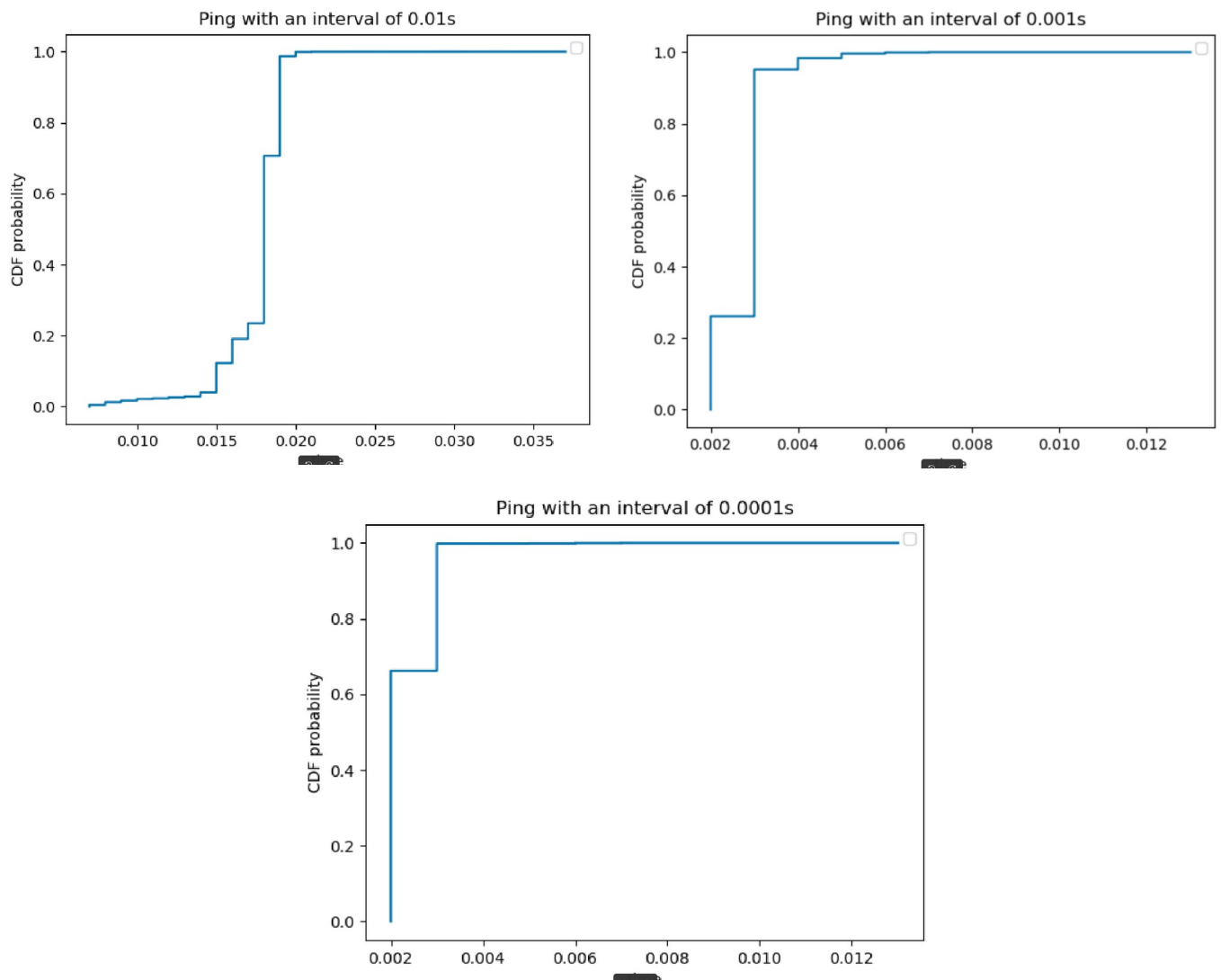
```
--- 127.0.0.1 ping statistics ---  
100 packets transmitted, 100 received, 0% packet loss, time 1092ms  
rtt min/avg/max/mdev = 0.008/0.016/0.048/0.004 ms
```

min and max are separated by 0.040ms, more than twice the mean value so this is a large range. The large max will be noise from context switches where the kernel temporarily schedules other tasks other than the ping command while it is running.

14. Lab machine to itself 10,000 times using flooding (-f)

```
--- 127.0.0.1 ping statistics ---  
10000 packets transmitted, 10000 received, 0% packet loss, time 79ms  
rtt min/avg/max/mdev = 0.002/0.002/0.028/0.000 ms, ipg/ewma 0.007/0.002 ms
```

15. CDF's of round trip times for different intervals



16.

From the above we can see the pattern that longer intervals seem to longer mean/ median round trip times. This is likely due to the fact that with shorter time periods, every ping hits the L1/L2 cache so look ups are cheap.

We can also see the shorter time period chart looks more angular as we only have a measurement precision of 3dp.

With longer intervals the pings have time to be evicted from the cache by other processes so kernel look ups are slower.

Performance

17.

we can massively increase the average delay measured by rdtme.c by increasing the system activity by running resource intensive background processes. One example of how this can be done is by using the stress -cpu command.

18.

not sure here...

19.

a single ping request to localhost has a round trip time of around 0.002ms (with flooding). The average TSC time between reads from rdtme.c was 52ns. So in 1 round trip, we can read TSC roughly $2/52e-3 = 38.46$ times.

20.

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ python3 matmul_slow.py
n=128 reps=2 checksum=107389.290000
avg cell time: 6.064707338993003e-06
avg calc time: 0.10108390101231635
```

```
def main(argv: list[str]) -> int:
    n, reps = parse_args(argv)

    #Initialise around a seed.
    a = init_matrix(n, 1.0)
    b = init_matrix(n, 2.0)

    c = zero_matrix(n)

    calc_times = []
    for _ in range(reps):
        start = time.perf_counter()

        cell_times = matmul_slow(a, b, c, n)

        end = time.perf_counter()
        elapsed = end - start

        calc_times.append(elapsed)

    print(f"n={n} reps={reps} checksum={checksum(c, n):.6f}")
    print(f"avg cell time: {mean(cell_times)}")
    print(f"avg calc time: {mean(calc_times)}")

    return 0
```

```
cell_times = []
# Intentionally simple O(n^3) matrix multiplication.
# This loop order is correct but cache-unfriendly for matrix
def matmul_slow(a: Matrix, b: Matrix, c: Matrix, n: int) -> list[float]:
    for i in range(n):
        for j in range(n):
            total = 0.0

            start = time.perf_counter()

            for k in range(n):
                total += a[i][k] * b[k][j]
            c[i][j] = total

            end = time.perf_counter()
            elapsed = end - start
            cell_times.append(elapsed)

    return cell_times
```


21.

```
ubuntu@ubuntu:~/CWM-FDI/assignment1$ python3 matmul_fast.py
n=128 reps=50 checksum=107389.290000
matmul_fast1 avg time = 0.086208896136377
matmul_fast2 avg time = 0.08872520921868272
matmul_fast3 avg time = 0.07195319108199329
```

fastest is matrix transposition method with row initialization. Row initialization adds each row to the cache so subsequent reads are much faster. The matrix transpose method is mathematically more efficient so the overall process is significantly faster than the others.

22.

to benchmark programs, run multiple iterations and calculate the mean, as noise/ fluctuations can cause execution times to vary wildly

make note of the CPU conditions under which a benchmark was run, ie. clock frequency/ number of cores, etc.

document code with comments clearly, making note of assumptions that you are making.